DRAFT — UNOFFICIAL — NOT FOR OPERATIONAL USE

CONCEPTS GUIDE

TM-40L

USAREUR-AF

CONCEPTS GUIDE — TM-40L COMPANION

SOFTWARE ENGINEER

HEADQUARTERS
UNITED STATES ARMY EUROPE AND AFRICA
(USAREUR-AF)
Wiesbaden, Germany

DRAFT — NOT FOR OFFICIAL USE. FOR TRAINING PLANNING PURPOSES ONLY.

12 MARCH 2026

DRAFT — UNOFFICIAL — NOT FOR OPERATIONAL USE

CONCEPTS GUIDE — TM-40L COMPANION

SOFTWARE ENGINEER

MAVEN SMART SYSTEM (MSS)

HEADQUARTERS, UNITED STATES ARMY EUROPE AND AFRICA Wiesbaden, Germany 2026

PURPOSE: This guide develops the mental models required to write, deploy, and maintain code on MSS effectively. It is a prerequisite companion to TM-40L and is intended to be read before beginning TM-40L task instruction.

DISTRIBUTION RESTRICTION: Approved for public release; distribution is unlimited.

TABLE OF CONTENTS

1. The Software Engineer's Role on MSS
2. The Foundry Execution Model — How Code Actually Runs
3. Idempotency as a Core Design Constraint
4. The Ontology as Your API Contract
5. TypeScript Functions — Extending the Ontology with Logic
6. OSDK — Building Applications Against the Ontology
7. Testing in a Platform Environment
8. Code as Operational Infrastructure — The Maintainability Imperative
9. Common SWE Failure Modes on MSS

DRAFT

SECTION 1 — THE SOFTWARE ENGINEER'S ROLE ON MSS

BLUF: The SWE is the engineer of record for production code on MSS. Your work forms the operational backbone of the data platform. Everyone else depends on it running correctly, every time.

MSS supports a tiered workforce. Understanding where the SWE sits in that tier structure is the first mental model you need.

The Foundry workforce tiers on MSS:

Tier	Designation	Primary Output	Code Level
TM-10	Maven User	Consumes data products, runs analyses	None
TM-20	Builder	Workshop applications, basic transforms, no-code pipelines	Minimal / drag-and-drop
TM-30	Advanced Builder	Complex pipelines, Contour datasets, data modeling	Light Python, no TypeScript
TM-40A	ORSA	Quantitative analysis, Commander products, statistical models	Python (analytical)
TM-40C	ML Engineer	Machine learning models, validation, deployment	Python (ML)
TM-40B	AI Engineer	AIP Logic, Agents, LLM integration	Python + prompt engineering
TM-40L	**Software Engineer**	**Production transforms, TypeScript FOO, OSDK applications**	**Python + TypeScript (production)**

The SWE is distinguished from every other role by one criterion: you write code that runs in production and that other people depend on. A TM-40A ORSA writes analytical Python. A TM-40C ML Engineer writes model training code. A TM-40L SWE writes the infrastructure layer — the scheduled transforms that power every downstream product, the TypeScript Functions that compute properties on demand for the Ontology, and the OSDK applications that expose MSS data to staff sections that have no Foundry access.

This distinction has an operational consequence. When an ORSA's analysis notebook produces an incorrect result, the ORSA revises it. When a production transform produces incorrect data, every analyst, every Workshop dashboard, and every staff section consuming that dataset is now working with bad information — and they may not know it. The SWE's failure mode is silent, systemic, and operationally consequential.

What a SWE does on MSS:

- Writes Python transforms that execute on the Foundry compute platform on a schedule or trigger

- Develops TypeScript Functions on Objects (FOO) that compute derived properties on Ontology objects at read time
- Builds OSDK-backed applications that give staff sections (V Corps G2, G3, G4, G6) structured programmatic access to MSS data
- Owns the code repository governance: branch management, peer review, CI gates, and promotion workflows
- Serves as the technical coordinator when builders and ORSAs need logic that exceeds their tier

What a SWE does NOT do on MSS:

- Write analysis notebooks as the primary work product — that is TM-40A
- Tune ML model hyperparameters — that is TM-40C
- Design AIP Agent chains — that is TM-40B
- Build Workshop applications through the GUI — that is TM-30

The boundary is not about capability. An SWE can do all of those things. The boundary is about primary responsibility and production ownership. The SWE owns what runs in production. Act accordingly.

SECTION 2 — THE FOUNDRY EXECUTION MODEL — HOW CODE ACTUALLY RUNS

BLUF: Python transforms are not scripts. They are compute jobs called by the platform with typed inputs and a typed output. Understanding this execution model prevents the most common class of production failures.

A script runs when you run it. It has access to your file system, your environment variables, your network, and your local state. You call it and it does something.

A Foundry transform is different. The platform calls it. The platform provides the inputs. The platform receives the output. The transform has no access to your local file system, no persistent state between runs, and no side effects outside of the defined output dataset. This is not a limitation — it is the design. It enables the platform to schedule, parallelize, cache, retry, and audit your code. But you must write code that fits this contract, or the platform will not behave as you expect.

The Foundry transform execution contract:

```
Input datasets (provided by platform)
          |
          ▼
  [your transform function]
          |
          ▼
Output dataset (consumed by platform)
```

Your function receives DataFrames (or Spark DataFrames) corresponding to your declared inputs. It returns a DataFrame (or writes to the output parameter). That is the entire interface. Everything else — environment variables, files, database connections, HTTP calls — is outside the contract and will behave unexpectedly or fail entirely in scheduled execution.

Mental model: treat every transform as a pure function

A pure function takes inputs, produces outputs, and has no side effects. Foundry transforms should be as close to pure functions as operationally possible. The closer to pure, the more predictable, testable, and debuggable your transform will be.

Deviations from purity that are operationally justified (reading a configuration parameter, calling an approved internal API) should be explicit, documented, and wrapped in error handling. Deviations that are not operationally justified are bugs.

What this means for how you write code:

Pattern	In a script	In a Foundry transform
Read a file from disk	`open("data.csv")`	**Not available.** Data comes from input datasets only.
Write a file to disk	`open("out.csv", "w")`	**Not available.** Data goes to output datasets only.
Persist state between runs	Module-level variable	**Not available.** Use @incremental with watermarks.
Log to console	`print(...)`	Available, but logs go to the platform build log.
Call an external API	`requests.get(...)`	Requires approved egress configuration. Not a default capability.
Use a random seed	`random.random()`	Will differ each run unless seed is fixed.

Vignette — V Corps Equipment Readiness Transform

A SWE is building a scheduled transform that ingests daily equipment status reports from a G4 feed and produces a cleaned, standardized readiness dataset. The initial implementation reads a date from an environment variable to filter the input. This works in local testing and in the notebook.

When the transform is promoted to production and scheduled, the environment variable is not set in the Foundry execution environment. The transform silently returns an empty DataFrame. Every dashboard downstream shows zero equipment — and the on-call analyst escalates to G4 at 0300 thinking a data feed is down.

The fix is not to set the environment variable in the execution environment. The fix is to not use environment variables in transforms. Parameterize using Foundry's dataset parameters or derive the filter value from the input data itself. The execution environment is the platform's, not yours.

SECTION 3 — IDEMPOTENCY AS A CORE DESIGN CONSTRAINT

BLUF: A transform that runs twice must produce the same result as a transform that ran once. This is the foundational contract of Foundry compute. Design every transform to honor it.

Idempotency means that the same inputs, run through the same logic, produce the same outputs — regardless of how many times the transform has run or when it was last run. A non-idempotent transform is operationally dangerous because Foundry may re-run transforms during builds, after failures, or when upstream data changes. If each run produces a different result, you have no reliable production dataset.

The idempotency test: ask yourself these questions before deploying any transform:

1. If this transform runs right now and then runs again in five minutes with the same input data, are the outputs identical?
2. If this transform's output dataset is deleted and the transform is re-run from scratch, does it produce the same result as an incremental update would have produced over time?
3. If I manually re-trigger this transform after a pipeline failure, will I get duplicates, data loss, or corruption?

If the answer to any of these is "no" or "I'm not sure," the transform is not idempotent and must be redesigned before it goes to production.

Common idempotency violations:

Violation	Description	Correct approach
Append without dedup	Each run appends new rows; re-runs create duplicates	Use a surrogate key + dedup on write, or use @incremental with a watermark
Timestamp-as-key	Using `datetime.now()` to generate a key makes every run unique	Derive keys from input data content, not execution time
Running counter	A transform that increments a counter each run	Store state in a dataset parameter or a Foundry-managed watermark, not in transform code
Order-dependent logic	Logic that assumes the output was in a particular state before this run	Design transforms to reconstruct output from inputs, not to modify existing state

NOTE

APPEND transactions are not inherently idempotent. If a transform re-runs, duplicate rows will be written unless you implement deduplication logic — typically via a content hash column and INSERT OR IGNORE pattern, or by using a surrogate key. For atomic full-dataset replacement, use SNAPSHOT transactions instead.

Incremental transforms and the idempotency corollary

Foundry supports incremental (@incremental) transforms that process only new or changed data since the last run. Incremental transforms are efficient but introduce a subtlety: the incremental output must be identical to what a full recompute would produce over the same data. If your incremental logic and your full-compute logic would produce different outputs for the same input history, one of them is wrong.

Test this explicitly during development: run the incremental transform over your test data, then wipe the output and run a full recompute, then diff the results. If they differ, you have an incremental logic bug. This test is not optional on transforms that feed readiness or operational data products.

Vignette — SITREP Aggregation Pipeline

A SWE builds an incremental transform that aggregates SITREP submissions by unit and date. The incremental logic correctly identifies new submissions. But the aggregation step uses a Python `dict` that accumulates over multiple runs via a shared module-level variable — a state persistence error. During a retry after a build failure, the variable retains values from the failed run. The output shows double-counted SITREP submissions for two battalions.

The correct design: every run reads only from the declared input datasets, computes aggregations from scratch over the relevant window, and writes a complete, consistent output. Module-level state is forbidden in production transforms.

SECTION 4 — THE ONTOLOGY AS YOUR API CONTRACT

BLUF: The Ontology is not a database schema you own. It is the API contract for the entire MSS data ecosystem. Treat changes to it with the same discipline you would apply to a breaking version change in a production REST API.

Every time a staff section opens a Workshop application showing equipment readiness, they are querying Ontology Objects. Every time Contour renders a dataset, it is reading Object properties. Every time an OSDK application in a V Corps staff section calls a TypeScript Function, it is invoking logic registered against an Object Type. The Ontology is the shared contract that makes all of this interoperate.

When you add a property to an Object Type, you are extending the contract — a non-breaking change. When you rename a property, remove a property, or change a property's type, you are breaking the contract. Every application, every Workshop module, every TypeScript Function, and every OSDK client that references that property will be broken.

The discipline for Ontology changes:

Change type	Risk	Required actions before making change
Add new property	Low	Document in Object Type description; notify downstream builders
Rename property	**Breaking**	Audit all consumers (Workshop, Contour, FOO, OSDK); migrate all consumers in same branch; coordinate with data steward
Remove property	**Breaking**	Same as rename, plus confirm no active consumers before deletion
Change property type (e.g., string → integer)	**Breaking**	Same as rename; additionally verify all ingestion transforms that write the property
Add new Object Type	Low	Document; coordinate naming with C2DAO governance standards
Add new Link Type	Low-medium	Audit if new link changes Object Set composition in Workshop
Change primary key	**Critical**	Full audit of all consumers; coordinate with C2DAO; treat as a major version change

The consumer audit process

Before making any breaking Ontology change, run a consumer audit:

1. Search all Workshop modules in your project for references to the property or Object Type
2. Search all TypeScript Function code for references to the property
3. Search all OSDK application code for references to the property
4. Search all Contour datasets derived from the Object Type

5. Search all Python transforms that write to the Object Type

This is not optional and it is not the Ontology team's responsibility — it is your responsibility as the SWE making the change. If you do not perform this audit, you will break production for users who have no visibility into your change.

The branch/promote workflow exists for this reason

MSS uses a branch/promote governance workflow. Changes to the Ontology in a dev branch do not affect production until the branch is promoted. This gives you a window to audit consumers, migrate them, and validate the full change in a dev environment before production impact. Use this window. The SWEs who bypass branch promotion under time pressure — "I'll just make the quick fix directly in production" — are the SWEs whose changes cause 0200 data outage calls.

Vignette — Equipment Object Type Property Rename

A SWE is cleaning up the Equipment Object Type. The property `eqStatus` has been informally renamed to `equipmentReadinessStatus` in conversations, and the SWE renames the property in the Ontology to match. The change is made on a Friday afternoon in a hurry before a long weekend.

Monday morning: the G4 readiness dashboard is blank. A Workshop module references `eqStatus` directly. A TypeScript Function that computes fleet-level readiness percentages also references `eqStatus`. Both are now broken. The SWE is not available. The on-call TM-30 builder does not know why the property name changed or what it was changed to.

A 10-minute consumer audit before the change would have identified both dependencies. A properly staged branch promotion would have caught the breakage in dev. Neither step was taken.

SECTION 5 — TYPESCRIPT FUNCTIONS — EXTENDING THE ONTOLOGY WITH LOGIC

BLUF: TypeScript Functions on Objects (FOO) add computed behavior to Ontology objects at read time. The primary design question is: should this logic run on a schedule and store a result, or run on demand and compute live?

A TypeScript Function (FOO) is a function registered against an Object Type that returns a computed value when called on an instance of that type. The function is not pre-computed — it executes when queried. This makes FOO appropriate for logic that must always reflect the current state of the object, but it imposes a cost: every query that invokes the function triggers a computation.

Stored result (transform) vs. computed on read (FOO) — decision framework:

Factor	Favor a scheduled transform	Favor a TypeScript Function (FOO)
Input data changes	On a schedule (daily, hourly)	In real time or continuously
Computation cost	High (expensive aggregation)	Low (simple arithmetic, lookup)
User needs	Scheduled freshness is acceptable	Must reflect current state at query time
Ontology consumers	Many consumers need the same result	Result is object-specific and contextual
Auditability	Required (stored result is auditable)	Not required (computed at read, no history)

A FOO that aggregates across thousands of objects or performs expensive joins will degrade query performance for all users of that Object Type — not just the users who explicitly invoke the function. Profile before deploying. If the computation is expensive, it belongs in a transform.

What TypeScript Functions can and cannot do:

TypeScript Functions operate within the Ontology context. They can: - Read properties of the object instance on which they are called - Navigate Links to related objects and read their properties - Call other registered FOO functions - Return primitive values, object references, or arrays

TypeScript Functions cannot: - Write to the Ontology (they are read-only) - Make external API calls - Access datasets directly (only via Ontology Objects backed by datasets) - Maintain state between invocations

The parameter design discipline

FOO functions that take parameters give users flexibility. FOO functions with many optional parameters become difficult to maintain and test. Design FOO functions with the minimum parameters needed to answer the business question. If you find yourself writing a FOO function that has five optional parameters and complex branching logic, consider whether this logic belongs in a transform instead.

Vignette — Equipment Readiness Calculation

The G4 staff section needs an equipment readiness percentage displayed on a per-unit basis in Workshop. Two approaches are considered:

Option A: A scheduled Python transform runs nightly, joins the Equipment dataset against the Unit dataset, computes readiness percentages, and writes results to a `UnitReadiness` Object Type. Workshop queries `UnitReadiness` directly.

Option B: A TypeScript FOO function is registered on the `Unit` Object Type. When called, it navigates the `UNIT_HAS_EQUIPMENT` Link, iterates all Equipment objects for that unit, and computes the readiness percentage on demand.

Option A is correct for this use case. Readiness percentages are computed from data that changes daily at most. The computation is moderately expensive over a large fleet. The result needs to be auditable (G4 needs to be able to explain a readiness figure as of a specific date). Option B would recompute for

every Workshop query, degrading performance for all Workshop users on the same Object Type, and would not support historical auditing.

Option B would be appropriate for a property that reflects real-time sensor data or a calculation that must be precise to the current moment — not for a daily aggregation.

SECTION 6 — OSDK — BUILDING APPLICATIONS AGAINST THE ONTOLOGY

BLUF: OSDK is the TypeScript API for building applications that access MSS data from outside Foundry. Understand the Ontology first. OSDK is how you access it — not what you are accessing.

The Ontology SDK (OSDK) is a generated TypeScript library that exposes the MSS Ontology to external applications. Staff sections that do not have Foundry access — or that need a custom application interface suited to their workflow — consume MSS data through OSDK-backed applications. A V Corps G2 analytical portal, a battalion maintenance tracking application, a readiness reporting tool embedded in a SharePoint site — these are all potential OSDK application consumers.

The OSDK mental model:

An OSDK application is an Ontology client. It does not talk to a database. It does not call a REST API you built. It queries Object Types, navigates Links, and executes Actions that are defined in the Ontology. The Foundry platform enforces access controls at the Ontology level — the OSDK application inherits whatever permissions the authenticated user (or service account) has been granted on each Object Type and Action.

This has a critical consequence: if the Ontology restricts an Object Type to read-only for most users, an OSDK application cannot bypass that restriction. The application does not own the data — the Ontology does. Design your OSDK application to work with Ontology permissions, not around them.

OSDK application architecture layers:

```
[ User / Staff Section ]
        |
        ▼
[ OSDK Application (TypeScript/React) ]
        |  queries Objects, navigates Links, calls Actions
        ▼
[ OSDK Client (generated TypeScript library) ]
        |  authenticated requests
        ▼
[ MSS Foundry Platform — Ontology API ]
        |  enforces CBAC, returns Objects
        ▼
[ Ontology Object Types, Links, Actions ]
        |  backed by
        ▼
[ Foundry Datasets (produced by transforms) ]
```

Each layer has a distinct responsibility. The OSDK application layer handles presentation and user interaction. The OSDK client layer handles authentication and API serialization. The Ontology layer handles access control and data modeling. The transform layer handles data production and freshness. A SWE who conflates these layers — for example, trying to write directly to a dataset from the OSDK application layer rather than through an Action — is bypassing the governance model.

Service account discipline

OSDK applications that run unattended (server-side applications, scheduled jobs, integration endpoints) authenticate using service accounts, not individual user credentials. Service account tokens are operational secrets. Apply the same handling discipline as a classified credential:

- Store in the C2DAO-approved credential store, not in code or config files
- Scope to least privilege — the service account should have read access only to the Object Types the application needs
- Rotate on a defined schedule and on any suspected compromise
- Log service account activity for audit purposes

Vignette — V Corps G3 Personnel Accountability Application

The V Corps G3 staff section needs a web application that shows real-time personnel accountability across assigned units. Personnel data lives in MSS as `Soldier` and `Unit` Object Types, with `UNIT_HAS_SOLDIER` Link Types backing the relationship. The G3 section has staff members who are not Foundry users.

The SWE builds an OSDK application: a React frontend that authenticates against the Foundry platform using the G3 staff members' individual credentials (SSO). The application queries `Unit` objects the user has access to, navigates `UNIT_HAS_SOLDIER` links to retrieve accountability data, and displays current status.

Because access control lives at the Ontology level, a G3 staff member can only see units they have been granted access to — no application-level access control logic is needed. The SWE does not build a separate permissions model. The Ontology is the permissions model.

SECTION 7 — TESTING IN A PLATFORM ENVIRONMENT

BLUF: You cannot run Foundry locally, but you can test most transform logic locally with the right shim strategy. Every production transform must have at least one passing unit test before it is eligible for branch promotion.

Testing Foundry transforms presents a challenge: the full execution environment — Spark cluster, Foundry dataset APIs, Ontology write paths — is not available on a local development machine. This tempts engineers to skip testing and rely on "it works in the notebook." This is a production failure waiting to happen.

The layered testing strategy closes this gap:

Layer 1 — Unit tests (local, no Foundry)

Write unit tests against the core transformation logic using mock DataFrames. Abstract your business logic into functions that take and return plain Python data structures (pandas DataFrames, dicts, lists). Test these functions with pytest. The Foundry execution wrapper calls these functions — the functions themselves do not need Foundry to run.

The minimum unit test bar for any production transform: - At least one test that validates the output schema (correct column names and types) - At least one test that validates correct output for a known input - At least one test that validates error handling for a malformed input

Layer 2 — Integration tests (Foundry dev environment)

After local unit tests pass, promote the transform to a Foundry dev environment and run it against dev-environment datasets. Validate that the transform runs to completion, that the output dataset has the expected row count and schema, and that no build errors or silent data quality failures occurred.

Integration tests in the dev environment catch issues that local unit tests cannot: Spark compatibility problems, Foundry API version issues, permission failures, and schema mismatches with live upstream datasets.

Layer 3 — Pre-promotion smoke tests (Foundry branch)

Before promoting a branch to production, run a smoke test on the full pipeline in the branch environment. The smoke test validates end-to-end behavior: the transform runs, the output matches expected characteristics, and downstream transforms or applications that depend on the output have not broken.

Automated smoke tests in CI are preferred. Manual smoke tests are acceptable for lower-velocity changes but must be documented in the branch promotion checklist.

Testing TypeScript Functions

TypeScript FOO functions are tested using Jest with a local Foundry OSDK mock. The test pattern: 1. Construct a mock object instance with known property values 2. Call the FOO function against the mock 3. Assert the returned value matches the expected result

Cover edge cases: null/undefined properties, empty link sets, boundary values for numeric computations.

The CI gate

The MSS code repository CI pipeline runs unit tests on every pull request. A branch cannot be merged if unit tests fail. This gate is not optional and not bypassable. Do not write tests that always pass without testing actual logic — this wastes the CI gate and creates false confidence. Write tests that will fail if the logic is wrong.

Vignette — Readiness Transform Test Coverage

A SWE writes a transform that computes a unit readiness score from equipment status codes. The transform is submitted for branch promotion without unit tests — "it ran successfully in the notebook."

During production promotion, the CI gate blocks the merge. The SWE writes a unit test overnight. In writing the test, they discover a logic error: the readiness scoring function assigns the wrong weight to a `NMC-P` (not mission capable — parts) status code, producing readiness scores 8-12% higher than the correct values. The notebook had not tested this case. The CI gate caught the gap. The transform was corrected before any commander received an inflated readiness product.

SECTION 8 — CODE AS OPERATIONAL INFRASTRUCTURE — THE MAINTAINABILITY IMPERATIVE

BLUF: Code you write on MSS will be maintained by someone else after you leave. Write for the next person. Unmaintainable production code is not technical debt — it is a readiness risk.

USAREUR-AF operates under rotation and PCS cycles. The SWE who built a production transform in January may be reassigned to a different installation by September. The contractor who wrote the OSDK application may transition off the contract in six months. The person who replaces them will inherit your code. They will have no tribal knowledge of the decisions you made, the edge cases you handled, or the upstream data quirks you worked around.

When that person cannot understand your code, one of two things happens: they do not touch it (the code calculates a risk), or they change it incorrectly and break production. Both outcomes are operational failures.

The maintainability checklist — apply before every pull request:

Item	Standard
Variable names	Descriptive, unambiguous — `unit_readiness_score`, not `urs` or `x`
Function names	Verb + noun, describe the action — `calculate_equipment_availability`, not `calc_ea` or `process`
Inline comments	Present on every non-obvious logic block — military-specific business rules especially
Module docstring	Present at the top of every Python module and TypeScript file — purpose, inputs, outputs, caveats
README	Present in every code repository — architecture, dependencies, how to run locally, how to promote
No magic numbers	Constants named and defined at module level — `NMC_STATUS_CODES = ["NMC-M", "NMC-P"]`, not `if status in ["NMC-M", "NMC-P"]` scattered through the code
No clever tricks	If a line requires a comment to explain why it is written that way, consider rewriting it to be self-explanatory
Dependency documentation	All external dependencies in `requirements.txt` or `package.json` with pinned versions

The Army Writing Style parallel

Army doctrine uses short sentences, active voice, and direct language because the reader may be reading under stress, with incomplete context, in a degraded environment. Apply the same principle to code. Code that is read at 0200 during a production incident by someone who did not write it is exactly analogous to a field manual being read in poor conditions by someone who is not a subject matter expert. Write for that reader.

Comments on military-specific logic

MSS contains business logic that is specific to Army operations: readiness status codes (FMC, PMC, NMC-M, NMC-P), personnel accountability categories, SITREP submission timelines, CCIR thresholds. These concepts are not self-evident to a developer who comes from outside the military context. Comment them every time they appear. A comment that says `# NMC-P: Not Mission Capable - Parts (awaiting parts, can be returned to service)` adds 30 seconds to write and may save an hour of confusion for the next maintainer.

Vignette — The Undocumented Pipeline

A senior contractor SWE builds a complex equipment readiness pipeline over six months. The code works correctly. It is not documented, the variable names are terse, and the business logic for how PMC equipment is counted in readiness calculations is embedded in a conditional nested four levels deep with no comment.

The contractor transitions off the project. A new SWE joins and is tasked with adding a new equipment status category. They cannot determine from the code how the existing categories are weighted. They make a change that they believe is correct. The change subtly alters how NMC-P equipment is counted. The readiness scores shift by a small amount. No alarm is triggered because the change is within noise for most units. Three months later, during a readiness review, G4 identifies a discrepancy between MSS readiness scores and paper records. The investigation traces back to the undocumented logic change.

The undocumented pipeline is the root cause. The fix cost three weeks of forensic analysis.

SECTION 9 — COMMON SWE FAILURE MODES ON MSS

BLUF: The most common SWE failures on MSS are predictable and preventable. Recognize them before you replicate them.

This section catalogs the failure modes observed in MSS production environments and explains the design principle that prevents each one. These are not hypothetical — each pattern has occurred on an operational data platform and produced a detectable operational impact.

Failure Mode 1: Transforms that depend on external state

A transform reads a file from a network share, calls an internal Army API, or reads an environment variable set on a specific compute node. The transform works in testing because the external resource is available. In production, the external resource is unavailable, has changed, or is unreachable from the Foundry execution environment.

Prevention: Transforms must declare all inputs through Foundry's input dataset mechanism. No external state. No file system reads. No API calls that are not formally integrated through Foundry's approved egress configuration.

Failure Mode 2: Building without tests

The transform is developed in a notebook, verified visually, and promoted directly to production without unit tests. The first failure is detected in production, by users, during operations.

Prevention: Every transform has unit tests that run in CI before merge. The CI gate blocks promotion if tests fail. This is non-negotiable.

Failure Mode 3: Modifying shared Ontology resources without auditing downstream consumers

A SWE renames a property, changes a type, or removes a property from a shared Object Type without auditing Workshop modules, TypeScript Functions, OSDK applications, and other transforms that depend on it.

Prevention: Consumer audit before any breaking Ontology change. Use the branch/promote workflow to validate the full change in dev before production. Coordinate with downstream owners.

Failure Mode 4: Bypassing branch/promote governance under time pressure

An urgent fix is needed in production. The SWE makes the change directly in the production Ontology or dataset, bypassing the dev branch and CI pipeline. The fix introduces a secondary error. Because CI was bypassed, the secondary error is not caught. The secondary error causes a downstream failure.

Prevention: Branch/promote governance exists precisely for urgent situations. A five-minute bypass can produce a multi-hour outage. If the timeline genuinely does not permit the full workflow, escalate to the data steward and document the exception — do not unilaterally bypass.

Failure Mode 5: Code that works in the notebook but breaks when productionized

Notebook code often accumulates implicit state: variables set earlier in the notebook session, libraries imported in a previous cell, data cached in memory from a previous run. When the same logic is converted to a scheduled transform, the implicit state is gone and the code fails.

Prevention: Before promoting notebook logic to a transform, run the notebook from scratch with a fresh kernel in a clean environment. Then run it again to verify idempotency. Only then convert to a transform.

Failure Mode 6: Service account credential sprawl

OSDK application service account tokens are stored in application config files, committed to version control, shared over email, or left in plaintext in deployment scripts. A single credential compromise exposes all MSS data accessible to that service account.

Prevention: All service account credentials in the C2DAO-approved credential store. Zero credentials in code, config files committed to version control, or informal communication channels. This is a security protocol, not a coding convention — violations are reportable incidents.

Failure Mode 7: Unscoped TypeScript Functions degrading Ontology performance

A TypeScript FOO function is written to compute a complex aggregation across a large number of linked objects. It performs correctly on small test datasets. In production, it is invoked by Workshop for every object in a large Object Set, causing query timeouts and degraded performance for all Ontology users.

DRAFT

Prevention: Profile TypeScript Functions against production-scale data volumes before deploying. If the computation is expensive at scale, move it to a scheduled transform and store the result as a property.

Failure Mode 8: Implicit schema assumptions in transforms

A transform assumes the input dataset has specific columns in a specific order. An upstream change alters the schema. The transform fails silently — it does not crash, it just produces incorrect output because it is reading the wrong column.

Prevention: Validate input schema at the top of every transform. Assert that required columns are present and have the expected types before any transformation logic runs. Fail fast and loudly on schema violations — do not propagate incorrect data downstream.

SUMMARY — PRINCIPLES FOR THE MSS SOFTWARE ENGINEER

The nine sections of this guide reduce to a small set of operating principles. Apply these before writing a line of code:

1. **Your code runs in production.** Other people depend on it. Act accordingly.
2. **Transforms are pure functions called by the platform.** No external state. No side effects.
3. **Idempotency is not optional.** Two runs of the same transform on the same data must produce the same output.
4. **The Ontology is a shared API.** Breaking changes break everyone. Audit before changing. Use the branch workflow.
5. **TypeScript Functions run at query time.** Expensive computations belong in transforms, not in FOO.
6. **OSDK is an Ontology client.** Understand the Ontology first. Access controls live at the Ontology layer.
7. **Test before you promote.** Unit tests in CI. Integration tests in dev. Smoke tests before production.
8. **Write for the next person.** Descriptive names, inline comments, README, no clever tricks.
9. **Common failures are predictable.** Recognize the failure modes in Section 9 before you replicate them.

Continue to TM-40L for task-based instruction in OSDK development, TypeScript Functions, Python transforms, CI/CD workflows, and security compliance.

END OF CONCEPTS GUIDE — TM-40L COMPANION

DRAFT